

TCS302: Data Structures with C

Unit 1, Lecture 2

Data Structure Operations and ADT

How data structures become useful through operations and abstraction.

Mr. Manoj Kaushik

Department of Computer Science and Engineering



Where Are We in Unit 1?

We are moving from storing data to working with data.

Lecture 1
Basic terminology

Lecture 2
Operations + ADT

Lecture 3
Pointers + structures

- A data structure is not only a storage format.
- We must also define what operations are allowed.
- ADT helps us separate usage from implementation.

Question: Is a stack only a pile of data, or also a set of rules?

Learning Outcomes

By the end of today's lecture, students should be able to:

1

Identify common operations

3

Define Abstract Data Type

2

Explain why operation cost matters

4

Separate interface from implementation

What Is an Operation?

An operation is an action performed on stored data.

Create

Traverse

Search

Insert

Delete

Update

Access

- Different structures support these operations differently.
 - Choosing the wrong structure makes the program slower or harder.
-

Create and Initialize

Before using a structure, we must prepare it correctly.

```
int marks[5] = {72, 85, 91, 68, 77};
```

72**85****91****68****77**

- Creation allocates or reserves space.
- Initialization gives meaningful starting values.
- Uninitialized data may produce wrong results.

Traversal

Traversal means visiting every element exactly once.



```
for (int i = 0; i < n; i++) {  
    printf("%d ", marks[i]);  
}
```

- Used for printing, summing and scanning data.
- For n elements, each element is visited once.
- Time complexity is usually $O(n)$.

Quick check: Can traversal be $O(1)$ if all n elements must be visited?

Searching

Searching means finding an element with a given value or key.

12	25	37	49	58
----	----	----	----	----

Target = 49

- Linear search checks elements one by one.
- Binary search needs sorted data.
- The best method depends on data organization.

```
if (arr[i] == key)
    return i;
```

Insert, Delete and Update

These operations change the contents of a data structure.

Insert

- Add a new element
- May require shifting
- May require free space

Delete

- Remove an element
- May require searching
- May cause underflow

Update

- Modify an existing value
- Needs correct position
- Useful in records

Real example: changing a student's wrong marks is an update operation.

Why Operation Cost Matters

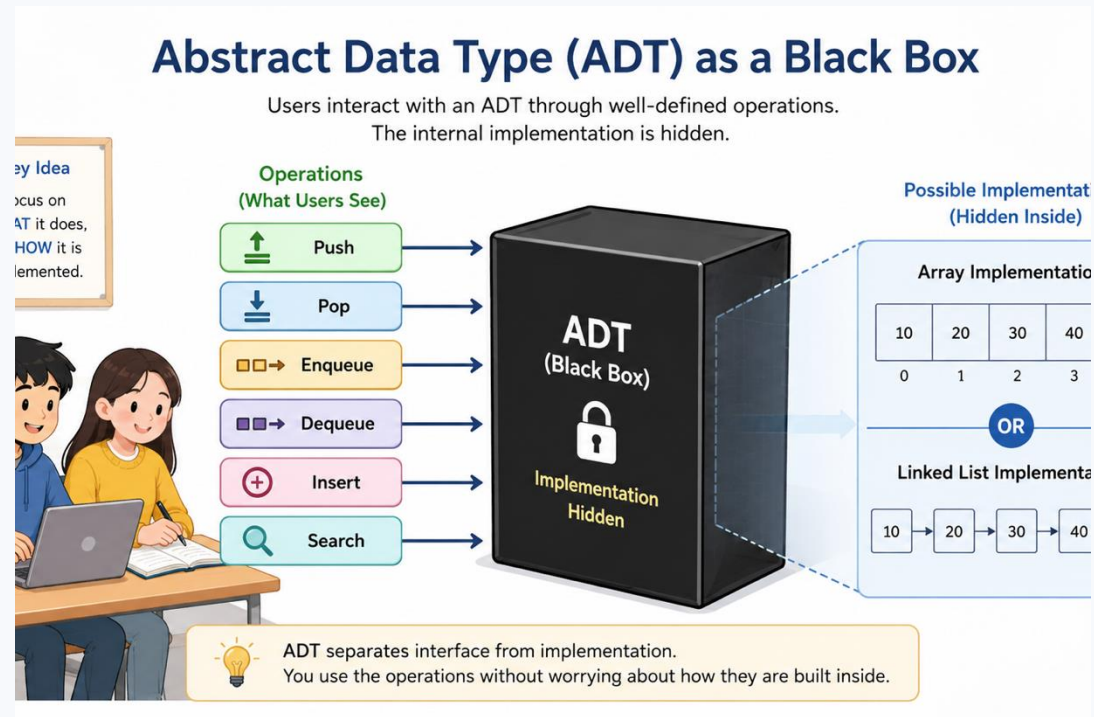
The same operation may be easy in one structure and costly in another.

Operation	Array	Linked List
Access by index	Fast	Slow
Insert at beginning	Costly	Fast
Traversal	Linear	Linear

Engineering decision: choose the structure according to the most frequent operation.

Abstract Data Type: Main Idea

ADT focuses on what can be done, not how it is implemented.



- The user sees operations.
- The internal representation is hidden.
- This makes programs easier to change and reuse.

Analogy: We use an ATM by pressing buttons; we do not see the internal banking code.

ADT = Data + Operations + Rules

A clear ADT description has three parts.

Data
What is stored?

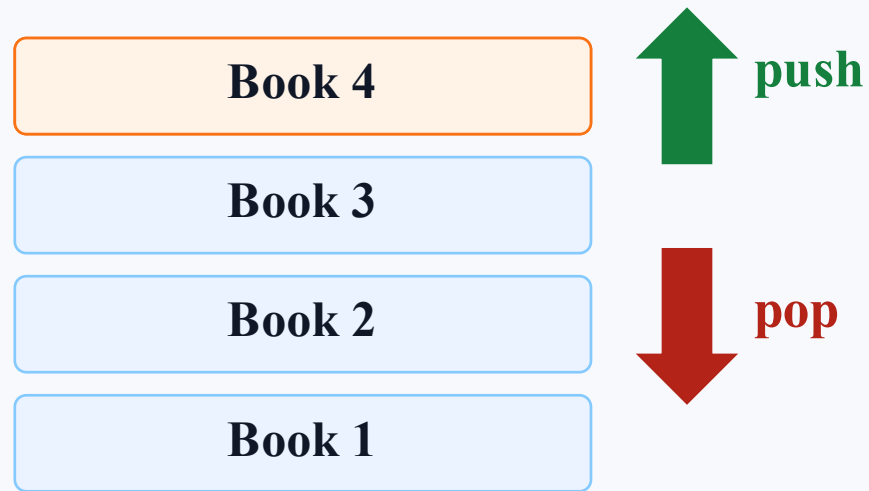
Operations
What can be done?

Rules
When is it valid?

Example: Stack ADT stores elements, supports push/pop/peek, and does not allow pop on an empty stack.

Stack as an ADT

A stack follows Last-In, First-Out behaviour.

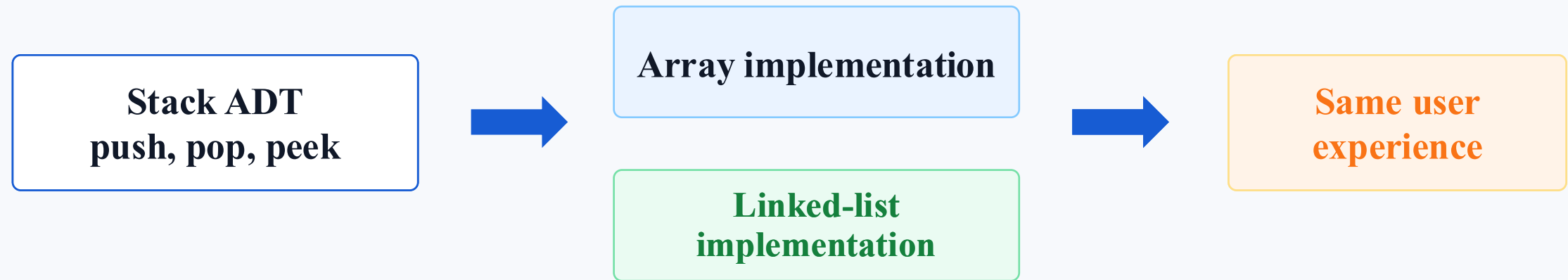


- `push(x)`: insert at top
- `pop()`: remove from top
- `peek()`: read top element
- Rule: pop is invalid when empty

Question: Why is the last book kept the first book removed?

ADT vs Implementation

The behaviour may remain same while storage changes.



Key idea: ADT is the promise; implementation is the method.

ADT Thinking in C

C can express ADT ideas using structs and functions.

```
typedef struct {  
    int data[100];  
    int top;  
} Stack;  
  
void push(Stack *s, int x);  
int pop(Stack *s);
```

- struct stores internal data.
- functions define legal operations.
- users should call functions, not directly modify top.

This is why pointers and structures are important in the next lecture.